Implement Symmetric Cryptography

AES (ECB and CBC) · Key Management · ChaCha20 Stream Cipher · Security Best Practices

From-scratch Python implementations verified against NIST/RFC test vectors and the production cryptography library — 18 of 18 tests passed

Project Name	Healthcare IAM Operations Simulation
Organization	Horizon Valley Medical Center (HVMC)
Rubric	Cryptography 1 — Implement Symmetric Cryptography (Semester 4, Week 10)
Code	`cryptography_1/symmetric_crypto/` : `aes.py` , `chacha20.py` , `key_management.py` , `test_and_demo.py` , `images/` , `results.txt`
Environment	Python 3.9.6 on Darwin arm64, 2026-10-13 15:34; cryptography package: 50.0.2
Prepared By	Sabitha S Karanavar
Report Date	October 13, 2026

Rubric Cross-Reference

Rubric requirement	Section	Evidence
Creation and documentation of a symmetric encryption implementation	2, Appendix	AES-128/192/256 written from scratch (FIPS 197), code excerpts, 18 passing tests
Working example of AES in ECB mode and CBC mode	3	Both modes with PKCS#7 padding; NIST SP 800-38A vectors; HVMC image, patient records and lab tokens encrypted in both modes
Demonstration of proper key management techniques	4	CSPRNG keys, PBKDF2 with salt, AES Key Wrap (RFC 3394), envelope encryption, key separation, KEK rotation and retirement
Implementation of 1 stream cipher	5	ChaCha20 (RFC 8439) from scratch; RFC vectors; nonce-reuse attack demonstrated
Documentation of security considerations and best practices	3.4, 6	Working attacks (ECB leakage, IV reuse, bit-flipping, padding oracle, two-time pad), mode comparison, HVMC best-practice checklist

1. Introduction

Symmetric cryptography uses the **same secret key** to encrypt and decrypt. It is fast enough to protect bulk data, which is why it protects nearly all ePHI at rest and in transit: Azure Storage encrypts `hvmcehrstorage` with AES-256, and TLS protects the patient portal with AES-GCM or ChaCha20-Poly1305. HIPAA lists encryption as an addressable safeguard for data at rest (§164.312(a)(2)(iv)) and in transit (§164.312(e)(2)(ii)). Properly encrypted ePHI is also exempt from breach notification under the HHS "unsecured PHI" guidance.

This project implements three things from scratch, using only Python's standard library: a **block cipher** (AES) with two **modes of operation** (ECB and CBC), a **stream cipher** (ChaCha20), and a **key management** layer. Each is

checked against official test vectors and against the production `cryptography` library (50.0.2). The cross-checks run automatically when that library is installed and are skipped otherwise.

2. AES Block Cipher Implementation

2.1 How AES works

AES (Rijndael, FIPS 197) encrypts one **128-bit block**, held as a 4×4 grid of bytes called the **state**, under a 128-, 192- or 256-bit key. It uses **10, 12 or 14 rounds**, and each round applies four steps:

Step	What it does	Mathematics / purpose
SubBytes	Replace every byte using the S-box	$S(x) = A \cdot x^{-1} \oplus 0x63$, where x^{-1} is the inverse in $GF(2^8)$ and A is a fixed bit-matrix (affine map). The only **non-linear** step, giving **confusion**.
ShiftRows	Rotate row r left by r bytes	Moves bytes between columns, giving **diffusion** across the state
MixColumns	Multiply each column by a fixed matrix	[2 3 1 1; 1 2 3 1; 1 1 2 3; 3 1 1 2] over $GF(2^8)$. An MDS matrix: changing 1 input byte changes all 4 output bytes.
AddRoundKey	XOR the state with this round's key	The only step that uses the key. Round keys come from the key schedule (RotWord, SubWord, Rcon = powers of 2 in $GF(2^8)$).

$GF(2^8)$ arithmetic. A byte is treated as a polynomial with bits as coefficients. Addition is XOR. Multiplication is polynomial multiplication reduced modulo the irreducible polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$ (0x11B). `xtime` multiplies by x: shift left, and XOR with 0x1B if bit 8 overflowed. Every non-zero byte has an inverse, computed here as a^{254}, because $a^{255} = 1$ in this 256-element field. The implementation **computes the S-box from this definition** instead of copying a table. The test confirms it reproduces the FIPS 197 values (S(0x00) = 0x63, S(0x53) = 0xED) and is a permutation of all 256 bytes.

```
def xtime(a):
    """Multiply by x (i.e. 2) in GF(2^8) modulo the AES polynomial x^8 + x^4 + x^3 + x + 1."""
    a <<= 1
    return (a ^ 0x11B) & 0xFF if a & 0x100 else a

def _build_sbox():
    sbox = []
    for x in range(256):
        b = ginv(x)
        # affine transform: s = b ^ rotl(b,1) ^ rotl(b,2) ^ rotl(b,3) ^ rotl(b,4) ^ 0x63
        s = b
        for i in range(1, 5):
            s ^= ((b << i) | (b >> (8 - i))) & 0xFF
        sbox.append(s ^ 0x63)
    inv = [0] * 256
    for i, v in enumerate(sbox):
        inv[v] = i
    return sbox, inv
```

```
def encrypt_block(self, block):
        s = add_round_key(list(block), self.round_keys[0])
        for r in range(1, self.rounds):
            s = add_round_key(mix_columns(shift_rows(sub_bytes(s))), self.round_keys[r])
        s = add_round_key(shift_rows(sub_bytes(s)), self.round_keys[self.rounds])
        return bytes(s)

def decrypt_block(self, block):
        s = add_round_key(list(block), self.round_keys[self.rounds])
```

```
        for r in range(self.rounds - 1, 0, -1):
            s = inv_mix_columns(add_round_key(inv_sub_bytes(inv_shift_rows(s)), self.round_keys[r]))
        s = add_round_key(inv_sub_bytes(inv_shift_rows(s)), self.round_keys[0])
        return bytes(s)
```

2.2 Verification

The implementation reproduces the **FIPS 197 Appendix C** vectors for all three key sizes, and the **NIST SP 800-38A** ECB and CBC vectors. It also produces **byte-identical output** to the production `cryptography` library for random keys and data. Encryption and decryption round-trip for every key size and for message lengths 0–49. The full test list is in Appendix A.

3. Working Example — AES in ECB and CBC Modes

A block cipher alone encrypts only 16 bytes. A **mode of operation** defines how to encrypt longer messages. Messages are padded with **PKCS#7**: add n bytes each of value n, where n is 1 to 16, so the length is a multiple of 16.

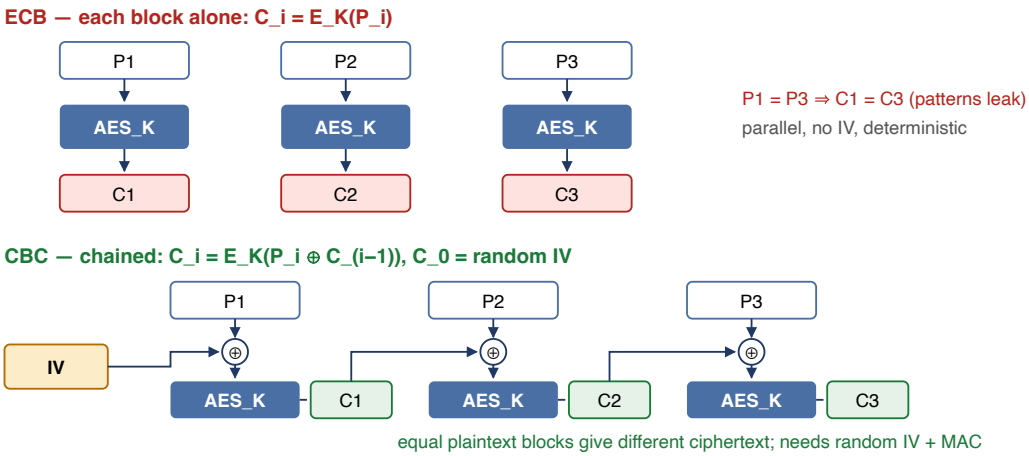


Figure 1 — ECB encrypts each block independently; CBC chains each block into the next, starting from a random IV.

```
def ecb_encrypt(key, plaintext, pad=True):
    """ECB: each block encrypted independently. Identical blocks -> identical ciphertext. INSECURE."""
    aes = AES(key)
    data = pkcs7_pad(plaintext) if pad else plaintext
    return b"".join(aes.encrypt_block(data[i:i + BLOCK]) for i in range(0, len(data), BLOCK))

def cbc_encrypt(key, plaintext, iv=None, pad=True):
    """CBC: C_i = E_K(P_i XOR C_{i-1}), C_0 = IV. A fresh random IV is generated and
    prepended to the output unless one is supplied (supplying one is for test vectors only)."""
    aes = AES(key)
    fresh_iv = iv is None
    iv = secrets.token_bytes(BLOCK) if fresh_iv else iv
    data = pkcs7_pad(plaintext) if pad else plaintext
    prev, out = iv, []
    for i in range(0, len(data), BLOCK):
        prev = aes.encrypt_block(bytes(a ^ b for a, b in zip(data[i:i + BLOCK], prev)))
        out.append(prev)
    return (iv if fresh_iv else b"") + b"".join(out)

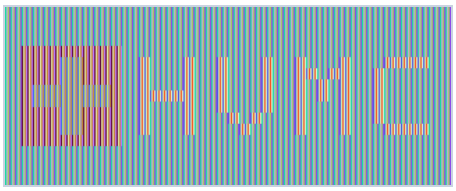
def cbc_decrypt(key, ciphertext, iv=None, pad=True):
    """P_i = D_K(C_i) XOR C_{i-1}. If iv is None, the first 16 bytes of ciphertext are the IV."""
    aes = AES(key)
    if iv is None:
        iv, ciphertext = ciphertext[:BLOCK], ciphertext[BLOCK:]
    if len(ciphertext) % BLOCK:
        raise PaddingError("ciphertext is not a whole number of blocks")
    prev, out = iv, []
    for i in range(0, len(ciphertext), BLOCK):
        block = ciphertext[i:i + BLOCK]
        out.append(bytes(a ^ b for a, b in zip(aes.decrypt_block(block), prev)))
        prev = block
    data = b"".join(out)
    return pkcs7_unpad(data) if pad else data
```

3.1 Demo 1 — The "ECB penguin" with the HVMC logo

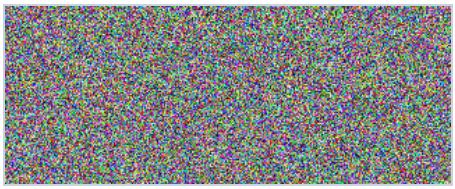
A 320×128 HVMC logo (10,240 AES blocks) was encrypted with the same AES-128 key in both modes, keeping only the image header in the clear:



Original — 4 distinct blocks



AES-ECB — still 4 distinct blocks: logo visible



AES-CBC — 10,240 distinct blocks: noise

ECB maps every identical 16-byte plaintext block (4 identical pixels) to the same ciphertext block. The colors change, but the **shape of the data is preserved**. CBC produced 10,240 distinct blocks out of 10,240, so no pattern survives.

3.2 Demo 2 — ECB leaks which patients share a diagnosis

Record	Diagnosis (plaintext)	ECB diagnosis block	CBC diagnosis block
Patient-0001	E11.9 T2 Diabet	`87a562144fa2f6fb…`	`81160194bcc48965…`
Patient-0002	I10 Hypertensio	`c331a36819daa4cd…`	`c644f946309884ef…`
Patient-0003	E11.9 T2 Diabet	`87a562144fa2f6fb…`	`ed590f89447da094…`
Patient-0004	J45 Asthma	`f709cfc53fbb7a64…`	`778a1d5d7eb87450…`
Patient-0005	E11.9 T2 Diabet	`87a562144fa2f6fb…`	`85e2711822241891…`
Patient-0006	I10 Hypertensio	`c331a36819daa4cd…`	`64e75a0ceacd086e…`

Six records produced only **3 distinct ECB diagnosis blocks**. Without the key, an attacker learns that Patients 1, 3 and 5 share a diagnosis, and if one record leaks, they learn all three. CBC hides this.

3.3 Demo 3 — CBC needs a fresh, unpredictable IV

Two different test results for the same MRN were encrypted with a fixed all-zero IV: **1 identical leading block**, revealing that the messages share a prefix. With a random IV (the default in `cbc_encrypt`, from `secrets.token_bytes`): **0 identical blocks**. The IV need not be secret, but it must never repeat and must be unpredictable.

3.4 Security limits of CBC (why encryption alone is not enough)

Demo 4 — Bit-flipping. In CBC, $P_1 = D(C_1) \oplus IV$, so flipping a bit of the IV flips the same bit of P_1. Without the key, an attacker changed an encrypted session token:

Original	`role=nurse;user=lisa.tran;ward=4B;exp=2026-10-20`
Forged	`role=admin;user=lisa.tran;ward=4B;exp=2026-10-20`
With MAC	**Same attack against the encrypt-then-MAC envelope (Section 4) detected: True**

Demo 5 — Padding-oracle attack. If a server reveals *whether padding was valid* (for example through different error messages), the attacker can decrypt everything. For each byte they adjust the previous ciphertext block until the padding is accepted, which reveals D(C)'s byte. The demo recovered `{"mrn":"HVMC-004417","dx":"F32.1`

`depression"}` (3 blocks) in **7,322 queries and 0.4 s**, without ever using the key. This is the class of attack behind POODLE and Lucky13 against TLS.

> **Lesson:** CBC gives confidentiality only. It must be combined with a MAC checked **before** decryption (encrypt-then-MAC), or replaced by an **authenticated** mode such as AES-GCM. The `cryptography` library's AES-GCM detected the same bit-flip with `InvalidTag`: True.

4. Key Management Techniques

Encryption is only as strong as the protection of its keys. `key_management.py` implements the NIST SP 800-57 key lifecycle for HVMC records:

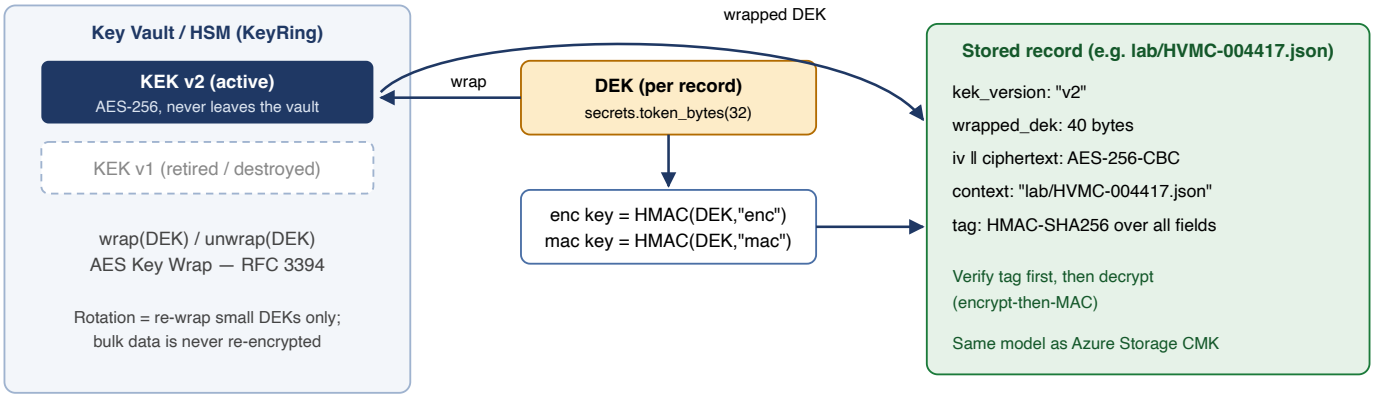


Figure 2 — Envelope encryption as implemented: per-record DEKs wrapped by a versioned KEK.

Technique	How it is implemented	Demonstrated result
Secure generation	`secrets.token_bytes` (OS CSPRNG). Never `random`, which is predictable (Mersenne Twister).	1,000 keys generated, all unique (test)
Password-based derivation	PBKDF2-HMAC-SHA256, 16-byte random salt, 600,000 iterations (OWASP)	139 ms per guess (≈7 guesses/s/core), **230,679× slower** than 1 iteration; same password + different salt → different keys
Key wrapping	AES Key Wrap (RFC 3394) built on the from-scratch AES; includes an integrity check	Matches RFC vectors and the library; a wrong KEK is detected
Envelope encryption	A fresh 256-bit DEK per record. Only the wrapped DEK is stored, next to the data.	3 lab/pharmacy records, each with its own DEK: True
Key separation	Encryption and MAC keys derived from the DEK with different labels; never one key for two purposes	Encrypt-then-MAC detects tampering
Context binding	The file path is included in the MAC	Record moved to another patient's file detected: True
Rotation	New KEK version; DEKs re-wrapped; ciphertext untouched	3 DEKs re-wrapped in 7 ms; bulk ciphertext unchanged: True
Retirement / destruction	Old KEK removed after re-wrap	New records decrypt: True; old v1-wrapped copies blocked: True

```
def derive_key_from_password(password, salt=None, iterations=PBKDF2_ITERATIONS, length=32):
    """Stretch a human password into a key. The salt makes every derived key unique and
    defeats precomputed (rainbow) tables; the iteration count slows down guessing."""
    salt = secrets.token_bytes(16) if salt is None else salt
    key = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, iterations, dklen=length)
    return key, salt

def envelope_encrypt(keyring, plaintext, context=b""):
    """Encrypt one record with its own DEK (AES-256-CBC + HMAC-SHA256, encrypt-then-MAC)."""
    dek = generate_key(256)
    enc_key, mac_key = hkdf_like(dek, b"enc"), hkdf_like(dek, b"mac")
    version, wrapped = keyring.wrap(dek)
```

```
    record = dict(kek_version=version, wrapped_dek=wrapped, iv_ct=cbc_encrypt(enc_key, plaintext), context=context)
    record["tag"] = _mac(mac_key, record)
    return record

def rewrap(keyring, record):
    """Key rotation: re-wrap the DEK under the active KEK. The bulk ciphertext is untouched,
    so rotating a KEK costs one small operation per record, not re-encrypting all data."""
    dek = keyring.unwrap(record["kek_version"], record["wrapped_dek"])
    mac_key = hkdf_like(dek, b"mac")
    new = dict(record)
    new["kek_version"], new["wrapped_dek"] = keyring.wrap(dek)
    new["tag"] = _mac(mac_key, new)
    return new
```

4.1 Applying this to HVMC's Azure environment

The Risk Assessment found **0 Azure Key Vaults** in the tenant, and `hvmcehrstorage` uses **Microsoft-managed keys**. Azure Storage already uses envelope encryption internally, with the same DEK/KEK model shown above. HVMC can take control of the KEK as follows:

Recommendation	Detail
Create a Key Vault with HSM-backed keys	Premium tier; enable **soft delete and purge protection** so a deleted key, which would make all EHR data unreadable, can be recovered
Customer-managed key (CMK) for hvmcehrstorage	Storage account managed identity gets only the "Key Vault Crypto Service Encryption User" role (wrap/unwrap), never key export
Rotation policy	Automatic KEK rotation every 12 months, or immediately after suspected compromise. Storage re-wraps automatically, as in Demo 8.
Separation of duties	Key administrators (Denise Coleman, Security) are not storage administrators (Fatima Ali, IT), so no single person holds both the key and the data
Logging	Send Key Vault audit logs to Log Analytics. This also helps close risk R01 from the Risk Assessment.

5. Stream Cipher — ChaCha20

5.1 How it works

A stream cipher generates a pseudorandom **keystream** from the key and a nonce, and XORs it with the plaintext: $C = P \oplus \text{KeyStream}(K, N)$ $P = C \oplus \text{KeyStream}(K, N)$ Encryption and decryption are the same operation. No padding is needed, so the ciphertext is exactly as long as the plaintext (73 bytes in Demo 6).

ChaCha20 (D. J. Bernstein, RFC 8439) builds a 4×4 state of 32-bit words: 4 constants ("expand 32-byte k"), 8 key words, a 32-bit block counter and 3 nonce words. It runs **20 rounds** (10 column rounds and 10 diagonal rounds) of the **quarter round**, which uses only **Add, Rotate and XOR (ARX)**:

$$\text{a += b; d ^= a; d <<<= 16} \;\cdot\; \text{c += d; b ^= c; b <<<= 12} \;\cdot\; \text{a += b; d ^= a; d <<<= 8} \;\cdot\; \text{c += d; b ^= c; b <<<= 7}$$

Addition mod 2^{32} provides non-linearity, and rotation and XOR spread bits. Finally the original state is added back so the rounds cannot be inverted. ARX uses no lookup tables, so ChaCha20 runs in **constant time** in software and resists cache-timing attacks. Table-based AES does not. This is why TLS offers ChaCha20 for devices without AES hardware.

```
def quarter_round(s, a, b, c, d):
    """The ChaCha quarter round: four add-rotate-xor steps on four state words."""
    s[a] = (s[a] + s[b]) & MASK32; s[d] = rotl32(s[d] ^ s[a], 16)
    s[c] = (s[c] + s[d]) & MASK32; s[b] = rotl32(s[b] ^ s[c], 12)
    s[a] = (s[a] + s[b]) & MASK32; s[d] = rotl32(s[d] ^ s[a], 8)
    s[c] = (s[c] + s[d]) & MASK32; s[b] = rotl32(s[b] ^ s[c], 7)

def chacha20_block(key, counter, nonce):
    """Produce one 64-byte keystream block.
    State (4x4 words): constants | key (8 words) | counter | nonce (3 words)."""
    if len(key) != 32 or len(nonce) != 12:
        raise ValueError("key must be 32 bytes and nonce 12 bytes")
    state = list(CONSTANTS) + list(struct.unpack("<8L", key)) + [counter & MASK32] + list(struct.unpack("<3L",
nonce))
    working = state[:]
    for _ in range(10):                      # 20 rounds = 10 x (column round + diagonal round)
        quarter_round(working, 0, 4, 8, 12)
        quarter_round(working, 1, 5, 9, 13)
        quarter_round(working, 2, 6, 10, 14)
        quarter_round(working, 3, 7, 11, 15)
        quarter_round(working, 0, 5, 10, 15)
        quarter_round(working, 1, 6, 11, 12)
        quarter_round(working, 2, 7, 8, 13)
        quarter_round(working, 3, 4, 9, 14)
    # add the original state back so the rounds cannot simply be inverted
    return struct.pack("<16L", *((w + s) & MASK32 for w, s in zip(working, state)))

def chacha20_xor(key, nonce, data, counter=1):
    """Encrypt or decrypt (identical operation) by XORing data with the keystream."""
    out = bytearray()
    for i in range(0, len(data), 64):
        ks = chacha20_block(key, counter + i // 64, nonce)
        out += bytes(a ^ b for a, b in zip(data[i:i + 64], ks))
    return bytes(out)
```

5.2 Verification and Demo 6 — Encrypting a lab message

The implementation matches the **RFC 8439** quarter-round and full-encryption test vectors, and is byte-identical to the `cryptography` library's ChaCha20. A lab message from the LIS to the EHR encrypted with a random 96-bit nonce

(`24d8f4d04c1571b7bfa307d4`) decrypted correctly: True.

5.3 Demo 7 — Nonce reuse destroys security (two-time pad)

If two messages use the same key and nonce, the keystream cancels out: $C_1 \oplus C_2 = (P_1 \oplus KS) \oplus (P_2 \oplus KS) = P_1 \oplus P_2$ An attacker who knows or guesses P_1 (a standard lab template) recovers P_2 exactly. The demo recovered `MRN HVMC-004418 HbA1c 9.8% HIGH`: True. A 96-bit random nonce makes accidental reuse negligible for up to about 2^{32} messages per key. Use a counter or rotate keys before that.

Broken example — RC4: the older stream cipher used in WEP and early TLS has biased keystream bytes, and RFC 7465 prohibits it in TLS. It is mentioned here only as a warning.

Aspect	AES-CBC (block)	ChaCha20 (stream)
Unit of encryption	16-byte blocks, needs padding	Byte-by-byte keystream, no padding
Main failure mode	Padding oracle, predictable IV, no integrity	Nonce reuse, no integrity
Side channels	Table lookups leak timing (use AES-NI hardware)	Constant-time by design (ARX)
Authenticated version	AES-GCM	ChaCha20-Poly1305 (RFC 8439)
Pure-Python speed (64 KiB)	211 ms	65 ms

6. Security Considerations and Best Practices

6.1 Choosing a mode

Mode	Confidentiality	Integrity	Notes	HVMC verdict
ECB	**Leaks patterns**	**No**	Demos 1–2. Acceptable only for a single random block (e.g. inside key wrap).	**Prohibited**
CBC	**Yes (random IV)**	**No**	Bit-flipping and padding oracle (Demos 4–5). Must be encrypt-then-MAC.	**Legacy only, with HMAC**
CTR / ChaCha20	**Yes (unique nonce)**	**No**	Nonce reuse is catastrophic (Demo 7)	**Only inside AEAD**
AES-GCM	**Yes**	**Yes**	Authenticated encryption; hardware-accelerated; 96-bit nonce must be unique	**Default choice**
ChaCha20-Poly1305	**Yes**	**Yes**	Authenticated; constant-time in software	**Approved**

6.2 HVMC symmetric-encryption best-practice checklist

#	Practice	Why (evidence in this report)
1	**Use authenticated encryption** (AES-256-GCM or ChaCha20-Poly1305)	Unauthenticated modes were forged (Demo 4) and decrypted (Demo 5)
2	**Never use ECB** for data	Logo and diagnoses leaked (Demos 1–2)
3	**Fresh random IV / unique nonce** for every message; never hard-code	Fixed IV leaked a shared prefix; reused nonce leaked a full message (Demos 3, 7)
4	**Generate keys with a CSPRNG** (`secrets`); derive from passwords only with a salted, slow KDF (PBKDF2 600k, scrypt, Argon2id)	Stretching made guessing 230,679× slower
5	**Envelope encryption with keys in Azure Key Vault / HSM**; never store keys in code, Git or next to the data	Section 4; tenant currently has 0 Key Vaults
6	**Rotate KEKs** at least yearly and after any compromise; retire old versions	Rotation took 7 ms with no data re-encryption (Demo 8)
7	**One key, one purpose** (separate encryption and MAC keys); bind context (record ID) into the tag	Moved record detected (Demo 8)
8	**Return identical errors** for any decryption failure; compare tags with constant-time functions (`hmac.compare_digest`)	Distinguishable padding errors enabled Demo 5
9	**Prefer AES-256** for long-lived ePHI	Grover's algorithm halves effective key length against a quantum attacker: AES-128 → 64-bit, AES-256 → 128-bit
10	**Use vetted libraries** (OpenSSL / `cryptography`), not custom code, in production	The from-scratch AES is 2,247× slower (211 ms vs 0.094 ms per 64 KiB) and not constant-time

6.3 Limitations of this implementation

- **Timing side channels:** the from-scratch AES uses table lookups (S-box, multiplication tables), whose timing depends on secret data. Production AES uses AES-NI / ARMv8 crypto instructions.

- **Key zeroization:** Python cannot reliably erase keys from memory (immutable bytes, garbage collection). HSMs and Key Vault avoid exposing keys to application memory at all.
- **Educational scope:** written to show the algorithms; correctness is proven by test vectors, but it has not been audited and is not intended for real patient data.

7. Conclusion

AES-128/192/256 was implemented from its mathematical definition and verified against FIPS 197 and NIST SP 800-38A. It produces identical output to the production library, and both ECB and CBC were demonstrated on HVMC data. ECB visibly leaked the HVMC logo and patient diagnosis groups; CBC hid them but was still vulnerable to bit-flipping and a padding-oracle attack that recovered a mental-health diagnosis in 7,322 queries. The key management layer showed CSPRNG generation, salted PBKDF2, RFC 3394 key wrap, envelope encryption, rotation and retirement, mirroring Azure Key Vault. ChaCha20 was implemented and verified against RFC 8439, and its nonce-reuse failure was demonstrated. The overall recommendation for HVMC is authenticated encryption (AES-256-GCM) with keys held in Azure Key Vault.

References

- NIST FIPS 197, *Advanced Encryption Standard (AES)* (2001, updated 2023).
- NIST SP 800-38A, *Recommendation for Block Cipher Modes of Operation* (2001); SP 800-38D (GCM).
- RFC 8439, *ChaCha20 and Poly1305 for IETF Protocols* (2018).
- RFC 3394, *Advanced Encryption Standard (AES) Key Wrap Algorithm* (2002).
- NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management* (2020).
- NIST SP 800-132, *Recommendation for Password-Based Key Derivation*; OWASP Password Storage Cheat Sheet (PBKDF2-HMAC-SHA256: 600,000 iterations).
- S. Vaudenay, "Security Flaws Induced by CBC Padding," EUROCRYPT 2002.
- RFC 7465, *Prohibiting RC4 Cipher Suites* (2015).
- 45 CFR §164.312(a)(2)(iv), §164.312(e)(2)(ii); HHS Guidance to Render Unsecured PHI Unusable, Unreadable, or Indecipherable.
- Microsoft Learn, *Azure Storage encryption for data at rest* and *Customer-managed keys for Azure Storage encryption*.

Appendix A — How to Run and Test Results

```
cd "cryptography_1/symmetric_crypto"
python3 test_and_demo.py            # 18 tests + 9 demos; writes results.txt, results.json, images/*.bmp
# optional cross-checks against the production library:
python3 -m pip install cryptography && python3 test_and_demo.py
```

Area	Test	What it checks	Result
AES	`test_cbc_random_iv`	Same plaintext encrypted twice gives different ciphertext	**PASS**
AES	`test_fips197_appendix_c`	FIPS 197 Appendix C vectors for AES-128, AES-192, AES-256 (encrypt + decrypt)	**PASS**
AES	`test_matches_cryptography_library`	Byte-identical to the production cryptography library	**PASS**
AES	`test_pkcs7`	Padding added correctly; malformed padding rejected	**PASS**
AES	`test_round_trip_all_key_sizes_and_lengths`	Encrypt→decrypt for 3 key sizes × 50 message lengths, ECB and CBC	**PASS**
AES	`test_sbox_from_maths`	S-box computed from GF(2^8) inverse + affine map: S(00)=63, S(53)=ED, bijective	**PASS**
AES	`test_sp800_38a_cbc`	NIST SP 800-38A F.2.1 CBC-AES128, 4 blocks	**PASS**
AES	`test_sp800_38a_ecb`	NIST SP 800-38A F.1.1 ECB-AES128, 4 blocks	**PASS**
ChaCha20	`test_matches_cryptography_library`	Byte-identical to the production cryptography library	**PASS**
ChaCha20	`test_rfc8439_encryption_vector`	RFC 8439 §2.4.2 'sunscreen' encryption vector	**PASS**
ChaCha20	`test_rfc8439_quarter_round`	RFC 8439 §2.1.1 quarter-round vector	**PASS**
ChaCha20	`test_round_trip`	Encrypt→decrypt for lengths 0, 1, 63, 64, 65, 1000	**PASS**
Key mgmt	`test_envelope_round_trip_tamper_and_rotation`	Envelope decrypts, detects tampering, survives KEK rotation, blocks retired KEK	**PASS**
Key mgmt	`test_generate_key`	1,000 CSPRNG keys all unique; correct lengths	**PASS**
Key mgmt	`test_key_wrap_matches_library`	Key wrap byte-identical to the cryptography library	**PASS**
Key mgmt	`test_pbkdf2_salt`	Random salt → different keys; same salt → same key	**PASS**
Key mgmt	`test_rfc3394_vectors`	RFC 3394 §4.1 and §4.6 AES key-wrap vectors	**PASS**
Key mgmt	`test_unwrap_with_wrong_kek_fails`	Wrong KEK detected by key-wrap integrity check	**PASS**

A.1 Full program output (results.txt)

```
HVMC Symmetric Cryptography — test and demo run
Python 3.9.6 on Darwin arm64, 2026-10-13 15:34; cryptography package: 50.0.2

=== Unit tests ===
test_cbc_random_iv (__main__.TestAES) ... ok
test_fips197_appendix_c (__main__.TestAES) ... ok
```

```
test_matches_cryptography_library (__main__.TestAES) ... ok
test_pkcs7 (__main__.TestAES) ... ok
test_round_trip_all_key_sizes_and_lengths (__main__.TestAES) ... ok
test_sbox_from_maths (__main__.TestAES) ... ok
test_sp800_38a_cbc (__main__.TestAES) ... ok
test_sp800_38a_ecb (__main__.TestAES) ... ok
test_matches_cryptography_library (__main__.TestChaCha20) ... ok
test_rfc8439_encryption_vector (__main__.TestChaCha20) ... ok
test_rfc8439_quarter_round (__main__.TestChaCha20) ... ok
test_round_trip (__main__.TestChaCha20) ... ok
test_envelope_round_trip_tamper_and_rotation (__main__.TestKeyManagement) ... ok
test_generate_key (__main__.TestKeyManagement) ... ok
test_key_wrap_matches_library (__main__.TestKeyManagement) ... ok
test_pbkdf2_salt (__main__.TestKeyManagement) ... ok
test_rfc3394_vectors (__main__.TestKeyManagement) ... ok
test_unwrap_with_wrong_kek_fails (__main__.TestKeyManagement) ... ok

----------------------------------------------------------------------
Ran 18 tests in 0.124s

OK
tests run: 18, failures: 0, errors: 0, skipped: 0

=== Demo 1: ECB vs CBC on an image (the 'ECB penguin' with the HVMC logo) ===
image 320x128, 10,240 AES blocks; distinct blocks: original 4, ECB 4, CBC 10,240
ECB keeps every repeated plaintext block repeated -> the logo stays visible. CBC output has no repeats.

=== Demo 2: ECB leaks equal values in patient records ===
Patient-0001  dx=E11.9 T2 Diabet  ECB dx-block 87a562144fa2f6fb…   CBC dx-block 81160194bcc48965…
Patient-0002  dx=I10 Hypertensio  ECB dx-block c331a36819daa4cd…   CBC dx-block c644f946309884ef…
Patient-0003  dx=E11.9 T2 Diabet  ECB dx-block 87a562144fa2f6fb…   CBC dx-block ed590f89447da094…
Patient-0004  dx=J45 Asthma       ECB dx-block f709cfc53fbb7a64…   CBC dx-block 778a1d5d7eb87450…
Patient-0005  dx=E11.9 T2 Diabet  ECB dx-block 87a562144fa2f6fb…   CBC dx-block 85e2711822241891…
Patient-0006  dx=I10 Hypertensio  ECB dx-block c331a36819daa4cd…   CBC dx-block 64e75a0ceacd086e…
ECB: 6 records but only 3 distinct diagnosis blocks -> an attacker sees who shares a diagnosis.
CBC: every block is different because of the random IV.

=== Demo 3: CBC needs a fresh random IV every time ===
fixed IV : 1 identical leading block(s) -> attacker learns the messages share a prefix
random IV: 0 identical blocks

=== Demo 4: CBC bit-flipping - changing ciphertext without the key ===
original : role=nurse;user=lisa.tran;ward=4B;exp=2026-10-20
forged   : role=admin;user=lisa.tran;ward=4B;exp=2026-10-20
same attack on encrypt-then-MAC envelope detected: True

=== Demo 5: padding-oracle attack decrypts CBC without the key ===
recovered plaintext: {"mrn":"HVMC-004417","dx":"F32.1 depression"}
correct: True; 7,322 oracle queries, 0.4 s, key never used by attacker

=== Demo 6: ChaCha20 stream cipher ===
nonce      : 24d8f4d04c1571b7bfa307d4
ciphertext : 517a6e58d40559d045fc814796cf1c0be52d44119ac63e10beddf8390e1697c0 … (73 bytes = plaintext length, no padding)
decrypted  : LIS -> EHR | MRN HVMC-004417 | Potassium 4.1 mmol/L | verified E. Petrova
first keystream block (64 bytes): 1d333d78f93b79950daea13bb6824e45c565125cd9eb0e20 …

=== Demo 7: nonce reuse turns ChaCha20 into a two-time pad ===
c1 XOR c2 == p1 XOR p2: True
attacker recovers p2 : MRN HVMC-004418 HbA1c 9.8% HIGH

=== Demo 8: key management ===
CSPRNG 256-bit key   : 7c382dec665f8f426bd3269f183b31d8 …  (secrets.token_bytes, os.urandom)
PBKDF2 600,000 iter   : 139 ms per guess on this Mac -> ~7 guesses/second/core
same password, 2 salts: fe8de5955c198318 vs 8cd6d8aaf8224e52 (different keys)
1 iteration          : 0.6 µs per guess -> stretching makes guessing ~230,679x slower
envelope encryption  : lab/HVMC-004417.json -> KEK v1, wrapped DEK ee516097dc15…, lab/HVMC-004418.json -> KEK v1, wrapped
DEK 45371e3fdc27…, rx/HVMC-004417.json -> KEK v1, wrapped DEK 72b2d84500a6…
rotate KEK v1 -> v2   : 3 DEKs re-wrapped in 7 ms, bulk ciphertext unchanged: True
retire v1             : new records decrypt: True; old v1-wrapped copies blocked: True
record moved to another patient's file detected (context bound into MAC): True

=== Demo 9: production comparison (cryptography package) ===
AES-256-CBC 64 KiB: from-scratch 211 ms vs cryptography 0.094 ms (2,247x slower)
ChaCha20   64 KiB: from-scratch 65 ms
AES-256-GCM (recommended): ciphertext+16-byte tag; bit-flip detected -> InvalidTag: True
```